

Informe Tarea 1

Gabriel Astudillo Muñoz, gabriel.astudillo@usm.cl

Hideki Nakamura, hnakamura@usm.cl

Vicente Rivera, vrivera@usm.cl

1. Parte 1

1.1. Diseño de la función lecturaSensor()

En la figura 1 se muestra la implementación de **lecturaSensor()**. Es una función que retorna un valor double aleatorio para simular lo que haría un sensor real, esto se hace usando el tiempo interno del sistema como referencia para generar los números. Por consiguiente, se puede trabajar con esos datos en el resto del código. Dicha función usa 16 Bytes en memoria.

Luego se implementó un enum de 3 casos para usar un modelo modular en la función main, pasando la mayoría del código original a **usarSensores()** pero separándolo en dichos casos, la separación puede observarse en la **figura 2**.

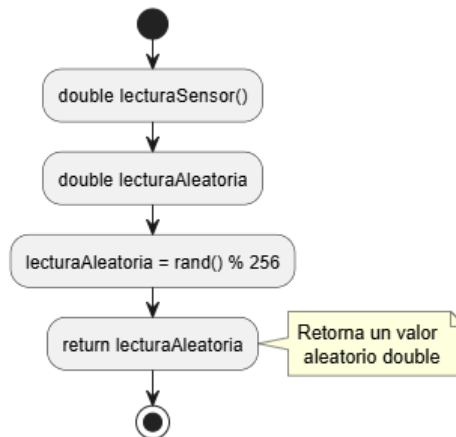


Figura 1, lecturaSensor()

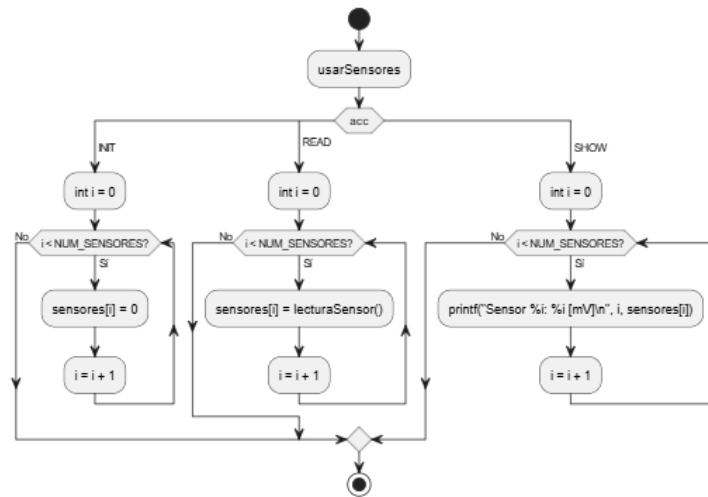


Figura 2, usarSensores()

1.2. Diseño completo de la nueva función main()

Según la figura 3, es posible observar que dada la implementación de código en formato modular el main queda de manera más ordenada y despejada. Opera de manera que toma como referencia el tiempo del sistema para generar números aleatorios y de esta forma generar los datos que la función **usarSensores** necesita para funcionar, después de esto imprime los resultados en pantalla.

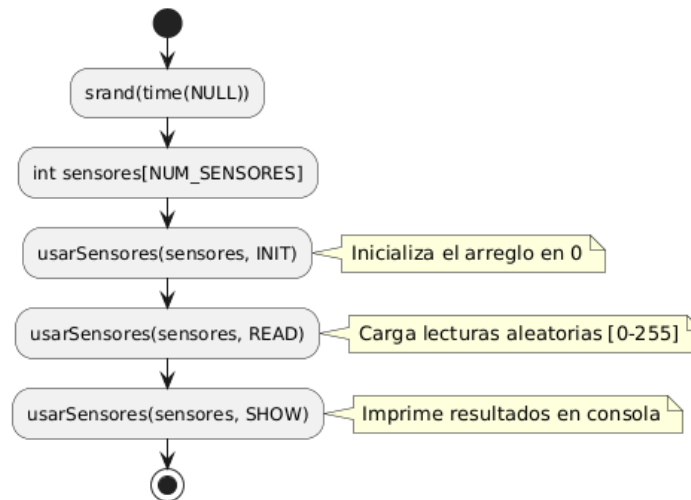


Figura 3, int main()

2. Parte 2

2.1. Hito 1

1) Qué es lo que retorna la función `clock()`.

La cantidad de tiempo que ha pasado el procesador en el código arroja estos datos en ticks. Los ticks que corresponde a un ciclo de reloj del procesador, es la unidad de tiempo más pequeña que el controlador del computador entiende.

2) Qué unidades tienen las variables `inicio` y `fin`.

La unidad de medida que tiene inicio y fin es de ticks, el cómo se mide cada tick puede cambiar dependiendo del sistema donde se usen las funciones.

3) Por qué para obtener el tiempo total de duración del ciclo `for` se debe dividir la diferencia entre `fin` e `inicio` con el valor constante `CLOCKS_PER_SEC`.

Se divide por `CLOCKS_PER_SEC` para pasar las dimensiones de fin e inicio de ticks a segundos, luego se multiplica por $1e9$ para pasar los datos a nanosegundos.

4) Explicar que hace `(double)` como prefijo de `(fin - inicio)`

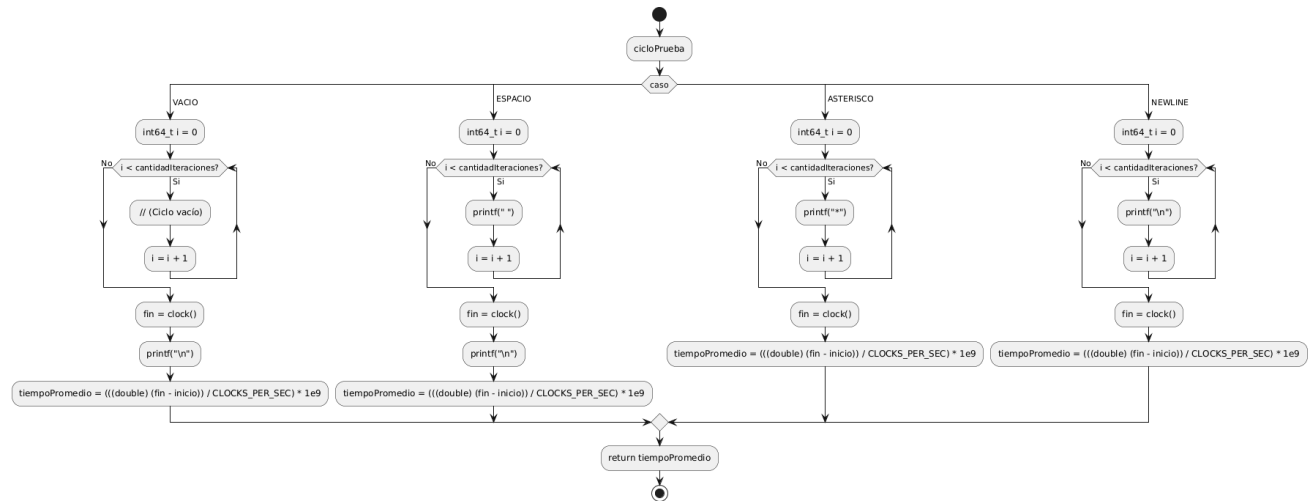
Le hace un cast o conversión explícita al resultado de `(fin-inicio)`, pasándolo de un `long int` a un flotante de precisión doble.

5) Explicar para qué sirve utilizar `(double)` como prefijo de `(fin - inicio)`

Permite mantener la precisión al evitar que se trunquen datos por hacer la división por la constante `CLOCKS_PER_SEC`.

2.2. Hito 2

En la figura 3 se muestra la implementación de **cicloPrueba()**. Es una función que retorna un valor `double` para medir el impacto temporal en nanosegundos de 4 tipos de ciclos `for` que se iteran $1e6$ veces. En primer lugar uno que no muestra nada en pantalla. En segundo lugar uno que enseña un espacio. En tercer lugar uno que enseña un asterisco. Y por último uno que enseña saltos de línea. Luego se implementaron un `enum` con los 4 casos (**VACÍO**, **ESPACIO**, **ASTERISCO** y **NEWLINE**) con fin de usar código modular en la función `main`, pasando la mayoría del código original a **cicloPrueba()** pero separándolo en dichos casos, esta separación puede verse en la figura 4.



[Figura 4. cicloPrueba\(\)](#) (zoom de la imagen)

2.3. Hito 3

A partir de los datos recolectados, destaca que el asterisco es el carácter que más demora en imprimirse repetidamente, con un tiempo promedio de 33.3665 s. En el extremo opuesto, el caso de vacío registró el mejor rendimiento con un promedio de apenas 0.0017 s. En cuanto a la consistencia del experimento, el espacio fue el que presentó mayor dispersión, alcanzando una desviación estándar de 5.2343 s. Estos datos se encuentran resumidos en la tabla insertada al final del documento. Para finalizar aquí están adjuntados las especificaciones de la máquina donde se realizaron las mediciones.

- Versión del compilador: Rev8/ by MSYS2 project 15.2.0
- Procesador Intel(R) Core(TM) i5-8265U CPU @ 1.60GHz 1.80 GHz
- RAM instalada de 12,0 GB (11,9 GB utilizable)
- Almacenamiento de 932 GB HDD WDC WD10SPZX-60Z10T0
- Tarjeta gráfica Intel(R) UHD Graphics 620 (128 MB)
- Tipo de sistema: Sistema operativo de 64 bits, procesador x64

Ciclos en [ns]	t1	t2	t3	t4	t5	t6	t7	t8	t9	t10
Vacío	10000 00	10000 00	10000 00	20000 00	30000 00	30000 00	10000 00	10000 00	30000 00	10000 00
Espacio	26831 00000 0	25881 00000 0	23818 00000 0	29996 00000 0	29266 00000 0	33311 00000 0	31858 00000 0	43134 00000 0	29611 00000 0	27221 00000 0
Asterisco	31049 00000 0	30007 00000 0	27195 00000 0	35118 00000 0	33766 00000 0	34858 00000 0	35156 00000 0	42400 00000 0	33509 99999 9	21767 00000 0
Newline	29751 00000 0	29481 00000 0	30585 00000 0	32890 00000	31540 00000 0	32268 00000 0	33792 00000 0	35505 00000 0	31919 00000 0	23073 00000 0

Ciclos en [s]	t1	t2	t3	t4	t5	t6	t7	t8	t9	t10
Vacío	0.001	0.001	0.001	0.002	0.003	0.003	0.001	0.001	0.003	0.001
Espacio	26.831	25.881	23.818	29.996	29.266	33.311	31.858	43.134	29.611	29.221
Asterisco	31.049	30.007	27.195	35.118	33.766	34.858	35.156	42.24	33.509	30.767
Newline	29.751	29.481	30.585	32.89	31.54	32.268	33.792	35.505	31.919	28.073

Categoría	Promedio [s]	Desviación estándar
Vacío	0.0015	0.00095
Espacio	29.9927	5.2343
Asterisco	32.4666	4.0197
Newline	31.5814	2.274